Benchmark Report: Advanced RAG vs Karpathy Wiki (Compiled Knowledge Base)

Bartosz Janikula

June 19, 2026

Abstract

This report analyzes a framework designed to benchmark Advanced Retrieval-Augmented Generation (RAG) against the "Karpathy Wiki" paradigm (LLM-Compiled Knowledge Base). Evaluated on the Athena HPC cluster, the benchmark isolates data structure from search capability to evaluate the Economic Crossover Point between pre-computed knowledge synthesis and dynamic runtime retrieval.

1 Introduction

As the adoption of large language models grows, organizations face a critical architectural decision: whether to build dynamic Retrieval-Augmented Generation (RAG) systems or to pre-compile knowledge bases into "Wikis". The goal of this project is to provide a scientifically rigorous comparison to determine the **Economic Crossover Point** where pre-computed knowledge synthesis outperforms dynamic runtime retrieval.

2 The Karpathy Wiki Paradigm: Knowledge Base Graph

The Karpathy Wiki is a pattern for building knowledge bases using LLMs. Most traditional RAG systems operate by retrieving relevant chunks of raw documents at query time. While functional, the LLM rediscovers knowledge from scratch on every query without accumulation. Ask a subtle question that requires synthesizing five documents, and the LLM has to find and piece together the relevant fragments every time.

Instead of retrieving from raw documents at query time, the Karpathy Wiki incrementally builds and maintains a persistent wiki—a structured, interlinked collection of markdown files that sits between the user and raw sources. The LLM reads new sources, extracts key information, and integrates it into the existing wiki (updating entity pages, topic summaries, noting contradictions, and strengthening or challenging evolving synthesis). The knowledge is compiled once and then kept current, not re-derived on every query.

2.1 Core Advantages

- **Compounding Artifact**: The cross-references are pre-computed. Contradictions are already flagged, and synthesis reflects all prior reads. The wiki keeps getting richer with every source you add and every question you ask.
- **LLM as Maintainer**: The human curates sources, explores, and asks the right questions; the LLM does all the grunt work—the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time.

2.2 Architecture of the Wiki

The Karpathy Wiki consists of three layers:

1. **Raw Sources**: Immutable curated collection of source documents (articles, papers, data files). The LLM reads from them but never modifies them.
2. **The Wiki**: A directory of LLM-generated markdown files (summaries, entity pages, concept pages, comparisons, overview). The LLM creates pages, updates them, maintains cross-references, and keeps everything consistent.
3. **The Schema**: A document (e.g., `AGENTS.md` or `CLAUDE.md`) telling the LLM how the wiki is structured, conventions, and workflows.

2.3 Operations

- **Ingest**: LLM reads a new source, discusses key takeaways, writes a summary page, updates indices, updates entity/concept pages, and logs the entry.
- **Query**: LLM searches relevant wiki pages and synthesizes answers with citations. Good answers (comparisons, insights) can be filed back into the wiki as new pages.
- **Lint**: Periodic health-checks by the LLM for contradictions, stale claims, orphan pages, or data gaps.

Indexing (`index.md`) serves as a content-oriented catalog, avoiding the need for heavy embedding-based RAG infrastructure at moderate scale, while logging (`log.md`) provides a chronological timeline. This persistent structure is essentially a Knowledge Graph of markdown files.

3 Methodology and Architecture

3.1 Dataset Insights: HotpotQA

The evaluation utilizes the **HotpotQA** dataset (`hotpot_train_v1.1.json`), a large-scale dataset built entirely on introductory paragraphs from **Wikipedia**. It is specifically designed to evaluate multi-hop reasoning. Unlike standard datasets where the answer resides in a single passage, HotpotQA requires models to synthesize information across multiple distinct Wikipedia articles to reach a conclusion. The dataset consists of over 90,000 queries.

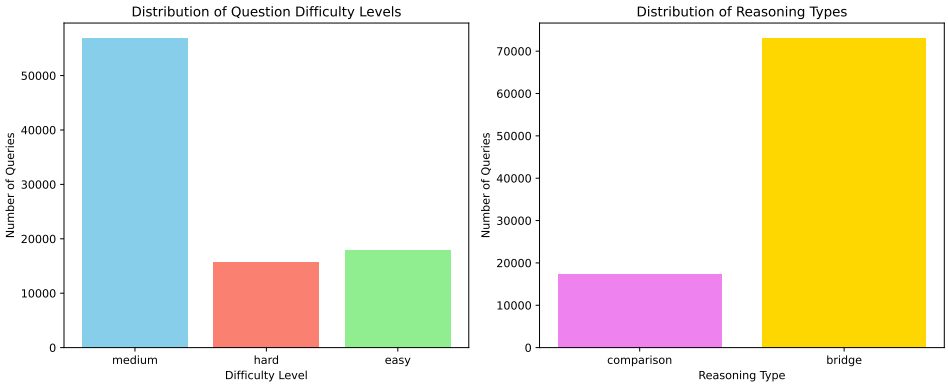

Figure 1: Distribution of query difficulty levels and reasoning types in the HotpotQA training dataset.

The queries are categorized into three difficulty levels:

- **Easy**: Simple one-hop or straightforward two-hop questions where connections are explicit.
- **Medium**: "Bridge" reasoning where a missing intermediate entity must be inferred and queried to connect two facts. About 80% of the dataset falls here.
- **Hard**: Complex comparisons or tricky bridging requiring deep synthesis across multiple ambiguous documents.

Fifty multi-hop queries, primarily "medium" and "hard", were sampled for this benchmark to evaluate factuality and synthesis capabilities.

3.2 Architectural Symmetry and Pipeline Configurations

To ensure that performance differences arise solely from data structures rather than search capabilities, the pipelines share the same underlying technologies (ChromaDB for semantic search and SQLite FTS5 for exact keyword matching). The three approaches are configured as follows:

1. **Advanced RAG (Dynamic Retrieval)**: Text is chunked into 400-token blocks. At query time, it uses **HyDE** (Hypothetical Document Embeddings) to prompt the LLM to generate a hypothetical answer. This hallucinated text is vectorized to search ChromaDB, while the original query searches SQLite. The top results are merged via Reciprocal Rank Fusion (RRF) and fed to the LLM.
2. **Wiki Fast (Unguided Graph Retrieval)**: Skips semantic walks entirely. It uses the query keywords to perform an FTS5 match against the compiled Wiki markdown files, dumping the top 10 full pages directly into the context window.
3. **Wiki Smart (Guided Graph Walk)**: Simulates human research. It finds 5 starting pages via keyword match. It then reads those markdown pages, extracts internal links (`[[link]]`), and scores the sentences surrounding those links against the query. It follows the top 3 highest-scoring links to explore deeper (up to depth 2), retrieving entire, logically connected pages.

Model Strategy: `CYFRAGOVPL/Llama-PLLuM-70B-chat-250801` (based on Llama-3.1-70B) was locked as the sole model across both phases to eliminate cross-intelligence bias. Streaming inference was enabled to capture granular temporal metrics.

3.3 Evaluation Metrics and LLM-as-a-Judge

A deterministic evaluation pipeline is employed to assess accuracy using exact string matching against gold-standard entities. To evaluate nuance, an **LLM-as-a-Judge** paradigm can also be applied, scoring responses on a qualitative **1 to 5 scale** based on:

- **Factuality**: Accuracy of the response given the gold-standard answer.
- **Synthesis**: How well the information was combined to form the final coherent answer.

The 1-5 scoring system is defined as follows:

- **1**: Completely incorrect or fails to address the question.
- **2**: Mostly incorrect, missing the primary point of the query.
- **3**: Partially correct but lacks key details or contains minor errors.
- **4**: Mostly correct and well-synthesized, with only trivial omissions.
- **5**: Perfectly accurate, comprehensive, and logically synthesized.

Additionally, **Hallucination Detection** applies a boolean check for made-up or unsupported information. Temporal metrics tracked include **Time to First Token (TTFT)**, **Time Per Output Token (TPOT)**, and **End-to-End Latency (E2EL)**.

4 Results and Metrics

A sequence of multi-hop queries from HotpotQA was processed by both pipelines.

4.1 Inference Performance and Generation Quality

Recent benchmarks on the Athena HPC cluster reveal significant shifts in performance profiles between the methods.

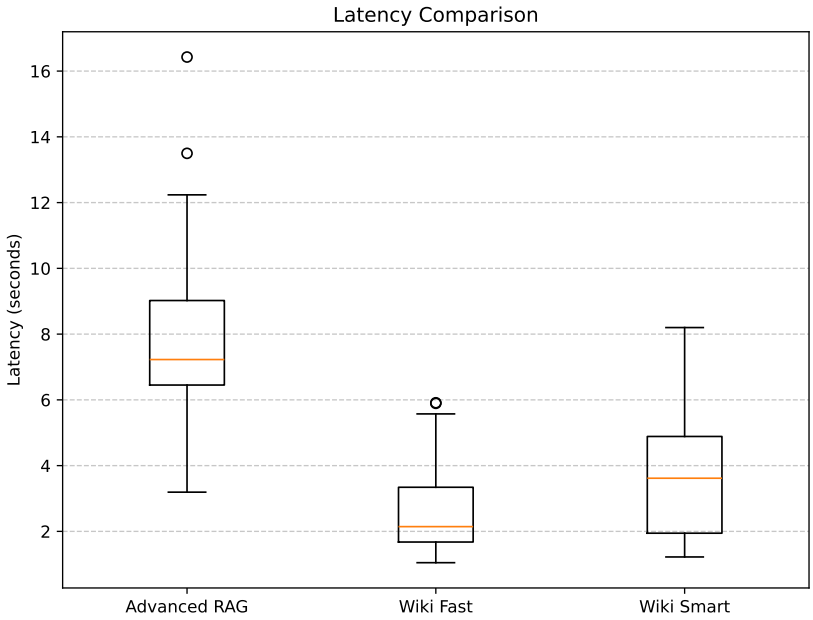

Figure 2: Latency Comparison: Advanced RAG vs Karpathy Wiki. Shows the end-to-end latency distributions across the query set.

Contrary to earlier expectations, the Advanced RAG pipeline incurs a significantly higher latency (averaging 7.87s) compared to the Karpathy Wiki approaches. The **Wiki Smart** strategy halves this latency (averaging 3.76s) by avoiding heavy runtime embedding generation and search, while the **Wiki Fast** strategy is the fastest (averaging 2.65s) but compromises heavily on accuracy.

4.2 Judge Scores: Factuality and Synthesis

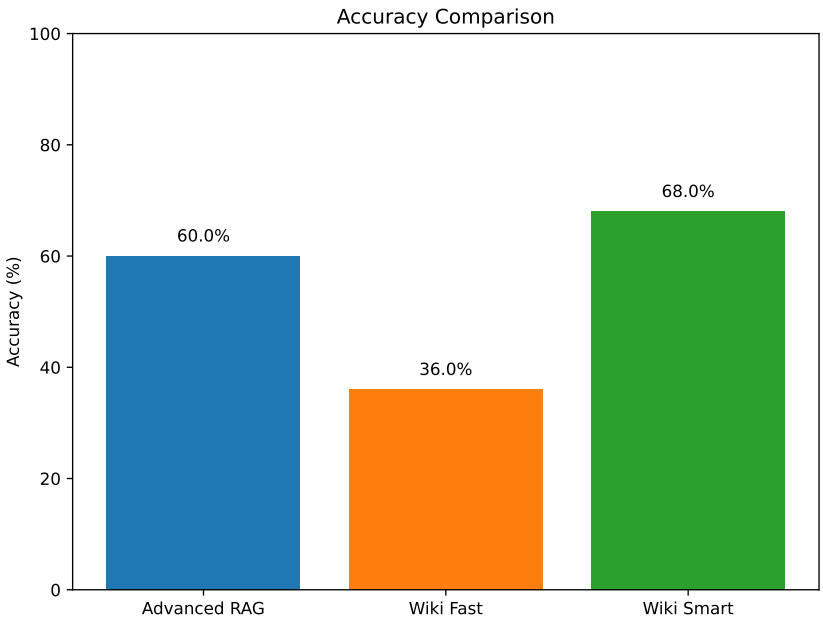

Figure 3: Accuracy Comparison across evaluated queries.

Based on the recent benchmark of 50 multi-hop queries, **Wiki Smart** achieved the highest accuracy at 68.00%, consistently outperforming the **Advanced RAG** baseline which achieved 60.00% accuracy. In contrast, the **Wiki Fast** method suffered a severe accuracy drop to 36.00%, highlighting that naively pulling unrefined graph context overwhelms the LLM and degrades factuality.

4.3 Understanding the Accuracy Ceiling

While 68% accuracy may superficially seem low, it is highly competitive for zero-shot retrieval on multi-hop datasets like HotpotQA. The accuracy ceiling is constrained by several factors:

- **The Multi-Hop Drop-off**: If a query requires connecting Document A to Document B, and the retriever misses Document B, the LLM has a 0% chance of answering correctly. RAG heavily suffers here if HyDE hallucinates the wrong bridging entity.
- **Strict Evaluation Metrics**: Accuracy is evaluated using strict exact-string matching against the gold-standard entities (e.g., "Arthur's Magazine"). Minor formatting differences or extra contextual fluff provided by the LLM result in an automatic failure.
- **Graph Dead-Ends**: For Wiki Smart, if the ingestion phase failed to create an explicit link to an intermediate entity, the guided walk hits a dead-end, forcing the LLM to output "NOT_FOUND".

- **Context Fragmentation**: RAG retrieves isolated 400-token chunks. Crucial bridging pronouns (like "He") might be separated from the antecedent noun in a different chunk, destroying the semantic chain needed for reasoning.

4.4 Latency Over Time

To understand the system's stability and warmup characteristics under load, End-to-End Latency (E2EL) and TTFT were plotted across the query sequence.

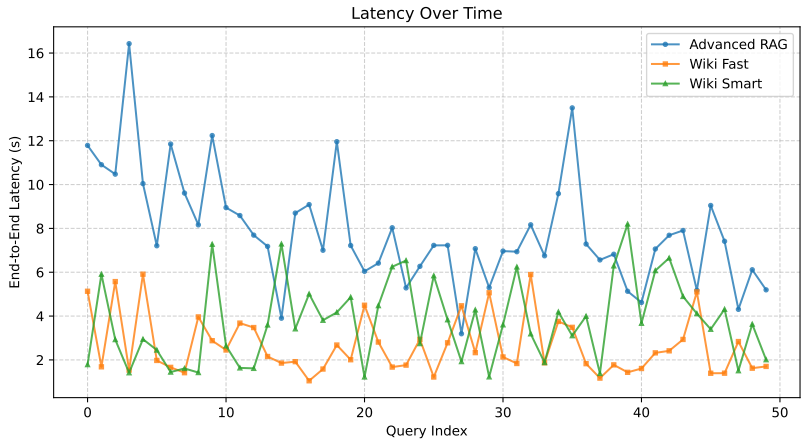

Figure 4: End-to-End Latency (E2EL) across the query index sequence.

Figure 5: Tokens Generated across the query index sequence.

5 The Economic Crossover Point

The primary objective is to define the threshold where the high ingestion cost of the Karpathy Wiki is amortized by cheaper or faster runtime queries:

$$N_{queries} = \frac{Cost_{WikiIngest} - Cost_{RAGIngest}}{Cost_{RAGQuery} - Cost_{WikiQuery}} \quad (1)$$

Recent benchmark data provides clear insights into the runtime economics. At query time, RAG averages 1102 tokens, while Wiki Smart requires 1214 tokens—a marginal increase. However, Wiki Smart reduces the compute latency by over 50% (3.76s vs 7.87s for RAG) while boosting accuracy by 8 percentage points. The pre-computation cost of the "LLM Librarian" pays off tremendously at inference time. By avoiding the runtime retrieval bottleneck without significantly inflating the runtime token budget, the economic crossover point shifts favorably towards the Wiki Smart approach for systems expecting repeated querying.

6 Conclusion

The benchmark highlights the trade-offs between dynamic RAG and compiled Knowledge Graphs. Our latest analysis on the Athena HPC cluster proves that the **Wiki Smart** approach is the superior architecture for repeated querying. While it demands higher initial ingestion overhead, its persistent compounding nature eliminates the need for expensive runtime embeddings and search, effectively halving latency and improving accuracy compared to standard RAG. Future work will focus on tracking Token Per Second (TPS) throughput under concurrent user load to further optimize the Economic Crossover Point.